REPÚBLICA PORTUGUESA
EDUCAÇÃO, CIÊNCIA E INOVAÇÃO

AEGP
AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

Programação e Sistemas de Informação

1

CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

MÓDULO 3

PROGRAMAÇÃO ESTRUTURADA

Programação Estruturada

AMBIENTE DE DESENVOLVIMENTO VISUAL STUDIO

SINTAXE DA LINGUAGEM C#

SUBPROGRAMAS (MÉTODOS)

Subprogramas - métodos



Um método é um **bloco de código**, a que é dado um nome, e que só **é executado** quando é "chamado". Também pode ser designado por **função**.

Utilizam-se para permitir a **reutilização de código**: construir o código uma vez e usar muitas vezes.

Permitem **resolver um programa** através de **pequenas partes** (os métodos ou funções).

Permitem **organizar** melhor o código.

```
static void somarDoisNumeros()
    {
        int x = 2, y = 4;
        Console.WriteLine(x + y);
    }
```

Subprogramas - métodos

Como "chamar" (executar) o método?

```
class Program
  {
     static void SomarDoisNumeros()
     {
        int x = 2, y = 4;
        Console.WriteLine(x + y);
     }

     static void Main(string[] args)
     {
        SomarDoisNumeros();

        Console.ReadKey();
     }
  }
```

Declarado dentro da classe Program

void significa que o método não devolve nenhum valor

O nome (identificador) deve **iniciar com maiúsculas**

É chamado **dentro do programa principal** (método **Main**) ou dentro de outro método.

Métodos com parâmetros (ou argumentos)

O método fica mais **genérico**.

Não fica "preso" a valores declarados dentro do seu código.

O objetivo é poder **reutilizar** em diferentes contextos.

```
class Program
  {
    static void SomarDoisNumeros(int a, int b)
    {
      Console.WriteLine(a + b);
    }

    static void Main(string[] args)
    {
      int x = 2, y = 4;
      SomarDoisNumeros(x, y);

      Console.WriteLine();
    }
  }
```

Os **dados** podem ser definidos no método através de **parâmetros**.

Os parâmetros dentro método comportam-se como variáveis.

Métodos com parâmetros (ou argumentos)

```
class Program
  {
     static void SomarDoisNumeros(int a, int b)
     {
        Console.WriteLine(a + b);
     }

     static void Main(string[] args)
     {
        int z, w;
        Console.WriteLine("Digite o 1º valor:");
        z = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Digite o 2º valor:");
        w = Convert.ToInt32(Console.ReadLine());
        SomarDoisNumeros(z, w);

        Console.ReadKey();
     }
  }
```

Ainda mais genérico.

O método é chamado com **dois valores pedidos ao utilizador** e "passados" para dentro do método.

Métodos com parâmetro de saída (função)

```
class Program
  {
     static int SomarDoisNumeros(int a, int b)
     {
        return a + b;
     }

     static void Main(string[] args)
     {
        int z, w;
        Console.WriteLine("Digite o 1º valor:");
        z = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Digite o 2º valor:");
        w = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine(SomarDoisNumeros(z, w));

        Console.ReadKey();
     }
  }
```

Neste caso, o tipo de dados do parâmetro de saída (**int**) é indicado em vez de “void”.

Utiliza-se a palavra **return** para devolver o valor calculado.